Lab 52: 40-field tool results push the review to its limit

Topic 5: Context Management & Reliability · Task 5.1: Manage conversation context to preserve critical information across long interactions

Runs on Simulator 5: Claude Code for Continuous Integration · Self-practice

Source: SkillCertPro practice exam Part 1, Q13 (unofficial third-party question)

The question

Long pull requests are causing the review step to run near the context limit. The trace shows each file-analysis tool call appends a large tool-result payload with build metadata, ownership data, full [unclear in transcript] output (possibly “lint” or “log”), historical CI runs, and dependency details, even though the reviewer uses only a small subset of those fields. What change best addresses the reliability issue?

A. Increase max_tokens for the review response and instruct Claude to ignore irrelevant tool-result fields.

B. Enable context editing to clear old tool results after the request crosses a configured threshold.

C. Change the tool wrapper to return compact, review-specific tool-result payloads with only the fields needed.

D. Apply prompt caching to the tool definitions and repeated review prefix to reduce context-window pressure.

Simulate it live

```
# In the browser: open Simulator 5, then Lab 52
cd labs
uv run serve.py

# In Claude Code, from the labs folder
/lab 52

# In a terminal, from simulators/simulator-5-ci-cd/
uv run casekit.py --case UC-13
```

What you should see

Verbose payload: tens of kilobytes. Compact payload: a few hundred bytes.

The verbose analysis payload is tens of kilobytes; the compact one is a few hundred bytes. Shaping the tool result is what keeps a long PR review inside the context window.

Open `simulators/simulator-5-ci-cd/evidence/` and find it yourself.

The answer

C. Wrap the tool so it returns only review-relevant fields.

Tool results become part of the conversation, and the context window holds messages, tool results, tool definitions and output. Every field a tool returns is carried forward, so a 40-field payload is not harmless just because the prompt says only a few fields matter. The most reliable fix is to shape the output before it becomes a tool_result . Prompt caching cuts cost and latency, but cached tokens still count toward the context window. Context editing can clear old results, but it does not replace compact current outputs.

Why the others fail

A. max_tokens caps the output and does not shrink the input. Telling Claude to “ignore” fields does not remove their tokens.

B. Treats the symptom later. Current payloads stay bloated, and needed older results may be cleared.

D. Cached tokens still count toward the context window.

Check yourself

- the compact payload is under 5% of the verbose payload
- the compact payload still carries the fields a reviewer uses (changed lines, functions, tests)
- learner can say why caching or a bigger max_tokens does not shrink the context the model carries

Now change one thing

Fix the loop in repo/stock/totals.py and the caller in repo/stock/jobs.py, rerun, and watch the blocking count fall.

Third-party practice material from SkillCertPro, reconstructed from a video transcript in the Tertiary Markdown Vault. Not a real exam item. Wording may differ from the original.